We create a simple play button and a slider for volume control. OnInput of this slider we call a JS function for changing the volume.The range for the slider is set to min 0 and max 100.

```
<input type="range" value="100"
max="100" min="0" oninput="Web
AudioDemo.changeVolume(this)" />
```

We have used bootstrap3 to style the web application.

Now that our context is ready, let us define a sequence of frequencies to play. This sequence shall be played on clicking the play button and while the sequence is playing, any change in the slider will reflect in the volume of the sound being played. The frequencies we selected for this are the general frequencies used by musicians all over the world (Trust us, we checked: http://dgit.in/NoteFrq).

Now let us store this in a JS array called sequence, as:

```
var sequence = [ 261.6, 293.7, 329.6,
349.2, 392, 440, 493.9,523.2, 523.2, 493.9,
440, 392, 349.2, 329.6, 293.7, 261.6];
```

Note: We are repeating the frequencies backwards as well for better impact. Feel free to try out your sequence of frequencies.

For the example that we have here, we shall use a duration of 0.7 seconds for each note and a speed of 0.35.

Now we need a buffer for our source. (Tip: You could use audio from <audio>, <video> tags straight into this) But we are going to create a custom buffer:

1. Get the sample rate

```
var rate = context.sampleRate;
```

2. Compute buffer length

```
var length = rate * duration;
```

3. Create the buffer

```
var buffer = context.create-
Buffer(1, length, rate);
```

4. Grab channel for inserting samples

```
var data = buffer.getChannelData(0);
```

5.Get the wave period

```
var period = Math.floor(rate / freq);
```

6.Generate white noise (random excitation)

```
for (var i = 0; i < period; i++) {
data[i] = Math.random() * 2 - 1;
}
```

*pointers

>>Interesting Web Audio developer videos

*Build an Instrument

>>A discussion on the true potential of Web Audio, while playing around with its features and options.

http://dgit.in/MusicWAud

*Future of Web Audio

>>Web Audio experts Chris Wilson (Google) and Chris Lowis (Web Audio Weekly) discuss the future of audio on the web.

http://dgit.in/WAudFuture

*Changing live audio

>>At JSConf 2016, Sam Bellen demonstrates how to alter audio from a live source using Web Audio API.

http://dgit.in/LiveWAud

*Creating UI for Web Audio API

>>At SwissJS 2015, Stephen Band discusses how to create UIs that control and reflect the state of the audio graph.

http://dgit.in/UIWebAud

7.Assign a Decay for feedback loop

```
var decay = 0.993;
```

8.Find Length of buffer beyond initial excitation

```
var len = data.length - period;
```

9.Feedback loop

```
for (var i = 0; i < len; i++) {
data[i + period] = (data[i] +
data[i + 1]) * decay * 0.5;
}
```

The Game that we are going to work with

Now let us create the audio source and assign our freshly created buffer to it:

```
var source = context.createBufferSource();
source.buffer = buffer;
```

Now as the penultimate step we connect our source to our gain node:

```
source.connect(node);
```

And finally we are ready to play:

```
source.start(startTime);
```

Clicking on the play button plays this sequence note by note in a loop:

```
for (var i = 0; i < sequence.length; i++) {
playNote(context, node, sequence[i],
startTime, duration);
startTime += speed;
}
```

For full example visit: http://dgit.in/WbAudio

Now for the second part of our example we are going to re-use most of our code and play a single frequency instead of looping through the entire array of frequencies. We will not delve deep into the code for developing the game as there is already an existing fiddle where you can play around with the code at http://dgit.in/BreakoutGame . Since we are concerned with generating audio on the fly therefore we shall only concentrate on the adding of audio part to this game.

Whenever we detect a collision within the Java Script function collisionDetection()